MQM — Multi-Quantum Mode Transmission

"Big ALU Quantum Jump: MQM-Based Ultra-Fast HBM Transfer with Low-Heat Architecture"

Transitioning CPU/GPU/NPU architectures from single-bit line processing to multi-dimensional MQM-based data transmission and processing.

💡 Core Technology: 3D-Mapping Transfer

A revolutionary data transmission method that maps information into a **3D Symbol Space** within a single cycle.

A single data symbol = (Frequency, Amplitude, Phase)

1-cycle data → 3D-mapping → High-speed transfer

$$S(t) = A \cdot \sin(2\pi f t + \phi)$$

Dimension	Role
Frequency (f)	Multi-carrier modulation (OFDM)
Amplitude (A)	Multi-level signaling (PAM/MQM)
Phase (φ)	Phase shift keying

📐 Core Formula

Data Transfer Throughput = [Frequency (n bits) + Amplitude (m bits)] × 2^p

Symbol	Description	Example
n	Bits encoded in the **Frequency** dimension	8 bits → 256 states
m	Bits encoded in the **Amplitude** dimension	8 bits → 256 levels
p	**Phase bits**	3 bits
2^p	**Phase states** (= Phase Number)	3 bits → **8 states**

Key Point: Phase is defined by **bit count (p)**, yielding **2^p independent phase slots**. With p=3: 3 phase bits → **8 phase states (0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°) → (n + m) × 2^p bits transmitted per cycle, per channel.**

📊 Phase bits → Phase states

Phase bits (p)	Phase states (2^p)
1	2
2	4
3	**8**
4	16
8	256

📊 Efficiency Example (n=8, m=8, p=3)

Dimension	Bits	States
Phase (φ)	3 bits	8 independent slots (0° ~ 315°)
Frequency (f)	8 bits	256 states per slot
Amplitude (A)	8 bits	256 levels per slot

Total capacity: (8 + 8) × 2^3 = 16 × 8 = 128 bits (16 Bytes) per cycle

---

🔧 Designer-Defined Bit Sequence Mapping

The assignment of n bits and m bits to the **leading** or **trailing** bit sequence is **freely chosen by the system designer**:

Option	Leading bits	Trailing bits
Option A	→ Frequency (n bits)	→ Amplitude (m bits)
Option B	→ Amplitude (m bits)	→ Frequency (n bits)

Key Point: MQM does **not** mandate a fixed mapping — the designer selects freely based on hardware constraints and optimization goals.

📜 MQM Design Freedom — 6 Configuration Options

In MQM, **Frequency (F), Amplitude (A), Phase (P)** can each serve as either **Data** or **State**.
The system designer chooses freely based on hardware environment.

Case 1: State × 1 + Data × 2 (3 options)

#	State (배율)	Data 1	Data 2	Formula of data Throughput
A)	Phase(z bit)	Frequency(n bits)	Amplitude(m bits)	[F(n) + A(m)] × 2^z ←**기본MQM**
B)	Frequency(n bits)	Phase(z bit)	Amplitude(m bits)	[Phase(z) + A(m)]
C)	Amplitude(m bits)	Frequency	Phase	[F(n) + Phase(z)]

⇨ **Best performance :** **case 1.** **A)**

→ ex) Data 1, Data 2 : 1st byte (Data 2) 2nd byte(Data 1) 등 적절히 바꿔서 설계가능

→ ex) Data 1, Data 2 : 1st byte (Data 1) 2nd byte (Data 2) 등 적절히 바꿔서 설계가능

Case 2: State × 2 + Data × 1 (6 options) **less efficiency**

#	State 1	State 2	Data	Formula of data Throughput
1	Phase(z bits)	Frequency(n bits)	Amplitude(m bits)	A(m) × 2^z
2	Phase(z bits)	Amplitude(m bits)	Frequency(n bits)	F(n) × 2^z
3	Frequency(n bits)	Amplitude(m bits)	Phase(z bits)	P(z)
4	Frequency(n bits)	Phase(z bits)	Amplitude(m bits)	A(m)
5	Amplitude(m bits)	Phase(z bits)	Frequency(n bits)	F(n)
6	Amplitude(m bits)	Frequency(n bits)	Phase(z bits)	P(z)

Case 3: Data × 3 (6 options)

#	Data 1	Data 2	Data 3	Formula of data Throughput
1	Phase(z bits)	Frequency(n bits)	Amplitude(m bits)	F(n) + A(m) + P(z)
2	Phase(z bits)	Amplitude(m bits)	Frequency(n bits)	F(n) + A(m) + P(z)
3	Frequency(n bits)	Amplitude(m bits)	Phase(z bits)	F(n) + A(m) + P(z)
4	Frequency(n bits)	Phase(z bits)	Amplitude(m bits)	F(n) + A(m) + P(z)
5	Amplitude(m bits)	Frequency(n bits)	Phase(z bits)	F(n) + A(m) + P(z)
6	Amplitude(m bits)	Phase(z bits)	Frequency(n bits)	F(n) + A(m) + P(z)

Key Point: There is no fixed rule — the designer selects the optimal configuration based on hardware constraints (GHz/THz, mv/v environment, heat budget, bandwidth target).

⚡ Why MQM Reduces Heat

Feature	**Benefit**
Multi-dimensional encoding	More bits per cycle → **fewer switching events**
Fewer switching events	→ **Less heat generated**
Lower heat	→ Ideal for **HBM, CPU/GPU internal buses**

📈 Channel Scaling: Quantum Jump Extension

Channels	**Register Size**	**Throughput gain (vs 64-bit CPU)**
64 ch	32 KB	4,096×
128 ch	64 KB	8,192×
256 ch	128 KB	16,384×
1,024 ch	512 KB	65,536×
4,096 ch	2 MB	262,144×

4,096 channels (n=8, m=8, p=8) achieves data capacity comparable to a **21-qubit quantum computer**.

📜 Implementation Pipeline

1. **Compression** — Raw data → Huffman Encoding
2. **Encoding** — MQM Mapping Table → Fourier Encoding (Superposition)
3. **Transmission** — Complex waveform over HBM/Data Channel
4. **Decoding** — Fourier Decoding → MQM Reverse Mapping
5. **Restoration** — Huffman Decoding → Raw Data for CPU/GPU

👤 Author & Credits

Role	Name	Contact
Inventor	Sungtae Park (scwpark)	scwpark@naver.com
Co-worker	Hyunuk Pak	iamgilchi@naver.com
Co-worker	EunHwan Park	dmsghks0421@gmail.com

- Zenodo : https://doi.org/10.5281/zenodo.19449687
- GitHub: https://github.com/scwpark/MQM-Wave-based-Computing-ALU-and-Data-Transfer
- Community: https://cafe.daum.net/scwpark

☕ Buy Me a Coffee

MQM is free and open-source forever — no patent, no fee. If MQM helped you, a cup of coffee keeps this old doctor-inventor going! 🦋

👉 Buy Me a Coffee

Or directly via Shinhan Bank (신한은행):

- 예금주: PARK SUNG TAE (박성태)
- SWIFT: SHBKKRSE
- Account: 110-623-275173

"I had this dream in the 1980s. Forty years later, I'm giving it to the world for free. A coffee would be nice though." ☕ 😄